

ATHENA AI MASTER SPECIFICATION MANIFEST (v2.0)

A Production-Ready Enterprise AI Operating System & Multi-Agent SaaS Platform Architecture

1. System Evolution & Enterprise Core Objectives

Athena AI (v2.0) transitions away from basic single-turn Q&A; chatbot frameworks into a unified **Enterprise AI Operating System Platform**. Built to resolve high knowledge fragmentation, scattered organizational document storage systems, manual operation loops, and data security exposure risks, the architecture establishes multi-user structural tenant separation, state-graph orchestration pipelines, and extensive hybrid semantic search indexing networks[cite: 1, 2].

2. Integrated Technology & Infrastructure Spectrum

Structural Dimension	Integrated Architecture Framework Component
API Routing & Gateway	FastAPI utilizing asynchronous loops (AsyncIO), WebSockets, and Server-Sent Events (SSE)[cite: 1, 2].
Orchestration Fabric	LangGraph cyclic graphs managing supervisor/worker execution loops and persistent checkpoints[cite: 1, 2].
Identity & Security	JWT, OAuth2, and Azure Entra ID providing strict Role-Based Access Control (RBAC)[cite: 1, 2].
Knowledge Core	Azure AI Search, PostgreSQL (pgvector), ChromaDB, and Neo4j Semantic Knowledge Graphs[cite: 1, 2].
LLM Infrastructure	Azure OpenAI Service, Anthropic Claude, DeepSeek, and local Ollama deployments[cite: 1, 2].
LLMOps & MLOps	MLflow prompt/evaluation tracking, Prometheus, Grafana, and input/output guardrails[cite: 1, 2].

3. Complete 56-Sprint Production Roadmap Roadmap

PHASE 1 — Foundation Platform (Sprints 1–8) [Status: Completed/Hardening]

- **Sprint 1 (Project Foundation):** Setup FastAPI structure, configuration management via Pydantic settings, standardized logging, and initial Docker multi-stage containers[cite: 2].
- **Sprint 2 (Authentication):** Establish secure JWT tokens, user sign-ups, password hashing via bcrypt, refresh tokens, and strict session isolation[cite: 2].
- **Sprint 3 (Document Management):** Build file processing endpoints handling PDF upload validation parsing, file systems, and core storage metadata metadata[cite: 2].
- **Sprint 4 (Enterprise RAG Core):** Deploy text chunk splitters, high-dimensional vector extraction, and local ChromaDB collection spaces[cite: 1, 2].
- **Sprint 5 (Conversation Engine):** Construct relational chat log history storage using SQLite tracking conversation indexes across sessions[cite: 1, 2].
- **Sprint 6 (Streaming Transmission):** Build asynchronous Server-Sent Events (SSE) token transmission loops with dynamic connection drop catching[cite: 1, 2].

- **Sprint 7 (Hybrid Search Fabric):** Fuse sparse token lookups (BM25) and dense semantic search matrices with metadata filtering and citation node mappings[cite: 1, 2].
- **Sprint 8 (Production API Hardening):** Implement global request/response error intercepts, Pydantic validation parameters, and complete unit testing coverage[cite: 2].

PHASE 2 — Enterprise AI Core (Sprints 9–16) [Status: Mostly Completed/Upgrading]

- **Sprint 9 (Tool Registry Framework):** Construct Python decorators to safely bind sandboxed utility libraries (Calculators, RAG, Web Scrapers)[cite: 1, 2].
- **Sprint 10 (Multi-Step Planner):** Develop an intent analysis compiler that breaks down complex user objectives into structural JSON planning blueprints[cite: 1, 2].
- **Sprint 11 (Agent Executor Loop):** Implement the primary reasoning execution thread (ReAct loops) mapping choices dynamically against expected tool outputs[cite: 1, 2].
- **Sprint 12 (Cognitive Memory Upgrade):** Refactor standard storage to decouple short-term message buffers, permanent conversation tables, and semantic profile caches[cite: 2].
- **Sprint 13 (Cross-Workspace Search):** Deploy multi-document search methods applying functional division permissions over document assets[cite: 2].
- **Sprint 14 (Context Ranking):** Build historical context scorers to automatically order historical entries and inject top relevance arrays into prompts[cite: 2].
- **Sprint 15 (Multimodal Expansion):** Embed vision transformer engines enabling OCR over dense non-textual corporate charts, data sheets, and flow maps[cite: 1, 2].
- **Sprint 16 (Operational Telemetry UI):** Construct primary evaluation views monitoring data limits, prompt cost patterns, and system exception rates[cite: 2].

PHASE 3 — Agentic AI Platform (Sprints 17–24) [Status: Planned]

- **Sprint 17 (LangGraph Integration):** Rewrite linear code chains into state-managed cyclic graph networks featuring explicit checkpoint execution rollbacks[cite: 2].
- **Sprint 18 (Supervisor Node Deployment):** Implement a primary root Supervisor routing agent that acts as an intent filter to orchestrate tasks to backend worker agents[cite: 2].
- **Sprint 19 (Research Agent Node):** Build specialized background nodes that execute multi-query internet scrapers, document validation, and summary reporting[cite: 2].
- **Sprint 20 (Document Intelligence Agent):** Program precise data extraction workers capable of pulling tabular relationships directly from large files[cite: 2].
- **Sprint 21 (Relational SQL Agent):** Deploy isolated read-only schema analytics agents that autonomously write, fix, and explain database query routines[cite: 2].
- **Sprint 22 (Sandboxed Code Agent):** Establish fully containerized execution micro-runtimes where agents securely generate and execute raw data manipulation scripts[cite: 2].
- **Sprint 23 (Workflow Orchestration Agent):** Construct continuous long-running background tasks handling deferred steps and direct webhook updates[cite: 2].
- **Sprint 24 (Multi-Agent Team Interaction):** Tune specialized peer messaging structures to support parallel workflows and cross-worker confirmation checks[cite: 2].

PHASE 4 — Enterprise SaaS Layer (Sprints 25–32) [Status: Planned]

- **Sprint 25 (Multi-Tenant System Isolation):** Implement organization ID filters across all tables, separating user data boundaries at the data block tier[cite: 2].
- **Sprint 26 (Corporate Directory Tools):** Develop admin toolsets handling secure employee invitations, branch assignments, and organizational mapping[cite: 2].
- **Sprint 27 (Granular RBAC Enforcer):** Create full access control gates parsing five core access profiles: Owner, Admin, Manager, Developer, and Viewer[cite: 2].
- **Sprint 28 (Stripe Commercial Integration):** Connect dynamic recurring subscription handling hooked into distinct access quota tiers[cite: 2].
- **Sprint 29 (Resource Usage Metering):** Code live telemetry log tracks recording compute token volumes, storage footprints, and calculation volumes[cite: 2].
- **Sprint 30 (Central Admin Control Board):** Build management panels showing global usage graphs, user logs, and global configuration toggles[cite: 2].
- **Sprint 31 (Asset Lifecycle Versioning):** Create document tracking modules tracking sequential workspace document edits, approvals, and collection rollbacks[cite: 2].
- **Sprint 32 (Agent Template Marketplace):** Design sharing directories for reuse of standard agent persona configurations and workflow blueprints[cite: 2].

PHASE 5 — Cloud Infrastructure & MLOps (Sprints 33–40) [Status: Planned]

- **Sprint 33 (Production Container Hardening):** Restructure core microservices into secure, minimal, non-root multi-stage Docker builds[cite: 2].
- **Sprint 34 (Managed Enterprise Cloud Shift):** Port local code components to cloud hosting, mapping files onto secure private object instances[cite: 1, 2].
- **Sprint 35 (Azure AI Search Migration):** Swap local indexing systems for enterprise Azure AI Search clusters supporting large-scale enterprise indexing[cite: 2].
- **Sprint 36 (High-Speed Redis Deployment):** Insert Redis blocks managing low-latency application token limits, API session memory, and common cache tracking[cite: 2].
- **Sprint 37 (Production PostgreSQL Migration):** Upgrade data persistence frameworks from local SQLite files into highly available cloud PostgreSQL clusters[cite: 2].
- **Sprint 38 (Declarative CI/CD Pipelines):** Deploy Git-driven GitHub Actions flows covering systematic code style testing, unit evaluations, and production push scripts[cite: 1, 2].
- **Sprint 39 (Observability Matrix Setup):** Integrate Prometheus collectors linked to Grafana setups logging network speeds, memory loads, and error logs[cite: 1, 2].
- **Sprint 40 (Systematic LLMops Framework):** Connect comprehensive MLflow setups monitoring prompts, testing configurations, and alternative models[cite: 2].

PHASE 6 — Enterprise Intelligence (Sprints 41–48) [Status: Planned]

- **Sprint 41 (Advanced OCR Engine Upgrade):** Build high-accuracy text parsers to pull bounding box data points cleanly from financial files and billing rows[cite: 2].
- **Sprint 42 (Neo4j Knowledge Graph Integration):** Setup cross-document semantic connection graphs tracking corporate entity rules inside a Neo4j framework[cite: 2].
- **Sprint 43 (Neural Re-ranking Pipeline):** Deploy Cross-Encoder modules within retrieval threads to optimize document contexts while saving token limits[cite: 2].
- **Sprint 44 (Durable NLP Service Core):** Create localized text utilities running intent profiling, summary parsing, and entities tracking[cite: 2].
- **Sprint 45 (Visual Prompt Studio):** Build user screens giving administrators side-by-side prompt testing capabilities and response evaluation metrics[cite: 2].
- **Sprint 46 (Safety Interception Guardrails):** Deploy real-time security systems checking fields for malicious code injections, masking PII, and checking context grounding[cite: 2].
- **Sprint 47 (Corporate App Connector Hub):** Construct native bridge connections into standard enterprise systems (Slack, Teams, Jira, SAP, Salesforce)[cite: 2].
- **Sprint 48 (Visual Workflow Automation canvas):** Build visual drag-and-drop web interfaces letting system admins map agent flows without custom code[cite: 2].

PHASE 7 — Production Scale & Launch (Sprints 49–56) [Status: Planned]

- **Sprint 49 (Production React Interface):** Port frontend frameworks over to optimized React / Next.js single-page tools styled via Tailwind CSS[cite: 2].
- **Sprint 50 (Analytics Dashboard Consolidation):** Expand tracking interfaces compiling cross-tenant active resource usage and financial reporting logs[cite: 2].
- **Sprint 51 (Distributed Dispatch Core):** Code resilient alert systems firing event pings through modern corporate channels based on health rules[cite: 2].
- **Sprint 52 (Resilient API Gateway Setup):** Place smart load balancers to balance user traffic across isolated microservice instances safely[cite: 2].
- **Sprint 53 (Celery Task Worker Grid):** Establish background distributed workers via Celery and RabbitMQ to handle large file array processes[cite: 2].
- **Sprint 54 (Performance Optimization Pass):** Fine-tune caching layouts, payload serialization rules, and compilation configs to minimize latency[cite: 2].
- **Sprint 55 (Enterprise SSO Integration):** Embed industrial SAML2 and OIDC interfaces enabling one-click enterprise sign-ons using Azure Entra ID[cite: 2].
- **Sprint 56 (System v1.0 Launch Deployment):** Finalize formal platform API document guides, developer software kits (SDKs), and push the stable build live[cite: 2].

4. Core PostgreSQL Multi-Tenant Database Schema

To guarantee complete tenant isolation before text vectors ever enter the high-dimensional query spaces, the system operates a relational ledger defining organization boundaries precisely[cite: 1].

```
CREATE TABLE organizations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(255) NOT NULL,  
  tier VARCHAR(50) CHECK (tier IN ('free', 'pro', 'business', 'enterprise')) DEFAULT 'free',  
  stripe_customer_id VARCHAR(255) UNIQUE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  hashed_password VARCHAR(255) NOT NULL,  
  role VARCHAR(50) CHECK (role IN ('Owner', 'Admin', 'Manager', 'Developer', 'Viewer')) DEFAULT 'Viewer',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE conversations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,  
  title VARCHAR(255) NOT NULL,  
  state_snapshot JSONB, -- Persistent state snapshots for LangGraph execution nodes  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```